ICMC USP SÃO CARLOS

USP Universidade de São Paulo



IA, Otimização e IoT para Indústrias e Negócios

Do Zero ao MLOps

OpAIoT
Optimization • AI • IoT

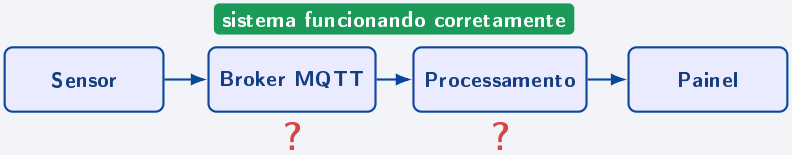
Monitoria

Observabilidade e Resiliência em Sistemas IoT

Roteiro

- A pergunta central
- Disponibilidade, confiabilidade e resiliência
- Observabilidade: logs, métricas e traces
- Estratégias de resiliência
- Arquitetura do laboratório
- Experimentos: broker e sobrecarga
- Síntese e encerramento

A pergunta que guia a aula



QUESTÃO

Nosso sistema IoT está funcionando corretamente. Mas o que acontece quando o **broker fica indisponível**, quando o **processamento não acompanha** a entrada de dados ou quando precisamos **recuperar** o serviço?

Não basta um sistema funcionar: ele precisa ser **observável**, **recuperar-se** de falhas e **preservar as garantias** esperadas dos dados.

Funcionar não significa funcionar sempre

Conceito	Significado	Exemplo
Disponibilidade	fração do tempo em que o serviço está operacional	broker acessível 99,9% do período
Confiabilidade	capacidade de cumprir sua função corretamente num intervalo	mensagens processadas conforme o esperado
Resiliência	capacidade de lidar com falhas e recuperar o serviço	cliente reconecta e retoma o processamento

$$A = \frac{MTBF}{MTBF + MTTR}$$

$MTBF$: tempo médio entre falhas

$MTTR$: tempo médio de recuperação

modelo p/ sistemas *reparáveis*

Disponível não é sinônimo de correto, atual ou completo.

Os três sinais de observabilidade

Logs

O que aconteceu: erros, desconexões, eventos pontuais.

Métricas

Quanto e quando: vazão, latência, tamanho da fila.

Traces

Por onde passou: o caminho de uma operação entre serviços.

Na prática de hoje, implementamos **logs** e **métricas**. *Traces* fica no conceito: o laboratório não implementa rastreamento distribuído, para manter a infraestrutura simples.

O que monitorar em IoT?

Métrica	Tipo	O que mostra
`iot_mqtt_connected`	Gauge	estado da conexão MQTT (0 ou 1)
`iot_generated_total`	Counter	medições geradas pelo publisher
`iot_received_total`	Counter	mensagens recebidas pelo consumer
`iot_processed_total`	Counter	mensagens efetivamente processadas
`iot_queue_depth`	Gauge	tarefas aguardando na fila interna
`iot_dropped_total`	Counter	mensagens descartadas (fila cheia)
`iot_end_to_end_latency_seconds`	Histogram	latência ponta a ponta (geração → fim)

Counter só cresce enquanto o processo vive (reinicia do zero a cada *restart*); *gauge* sobe e desce; *histogram* permite estimar a latência $p95$.

A escolha do tipo de métrica já é uma decisão de observabilidade.

O que fazer quando algo dá errado?

Mecanismo	Finalidade
Retry + *backoff*	repetir após falhas transitórias, sem martelar o serviço
Buffering	guardar dados enquanto o destino está indisponível
Idempotência	evitar efeito duplicado ao reprocessar a mesma mensagem
Backpressure	sinalizar o produtor para reduzir ou suspender o envio
Load shedding	descartar mensagens quando a fila atinge o limite

$$\frac{dQ}{dt} \approx \lambda(t) - X(t)$$

se a chegada λ supera a vazão X, a fila Q cresce

QUESTÃO

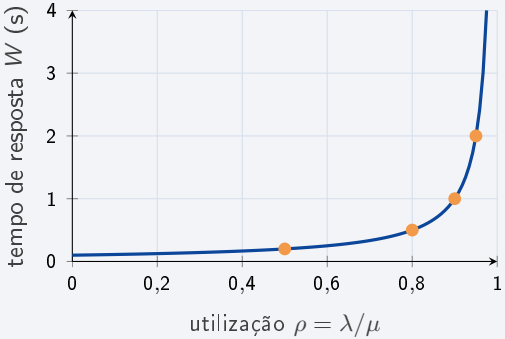
Se o consumidor processa 10 mensagens por segundo, mas chegam 30 por segundo, o que acontece após alguns minutos?

Utilização, lei de Little e saturação

$$\rho = \frac{\lambda}{c\,\mu}$$

$$L = \lambda\, W$$

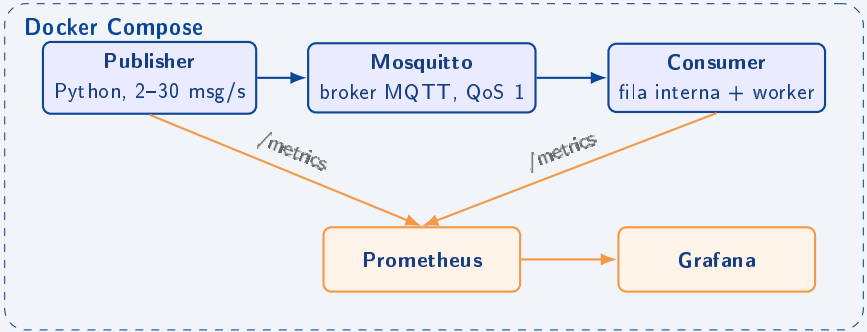




Na M/M/1, $W = 1/(\mu - \lambda)$: de $\rho = 0{,}5$ para 0,9, o **tempo de resposta** (espera + serviço) **quintuplica**.

W **só existe se** $\lambda < \mu$ **— o slide anterior tinha** $\lambda{=}30 \geq \mu{=}10$.

Arquitetura do laboratório



Todos os componentes executam no mesmo host; o consumer separa o `callback` MQTT do processamento, com uma fila interna limitada entre os dois.

QoS 1 garante a entrega da mensagem ao broker/consumer — não que a aplicação a processe: com a fila interna cheia, ela ainda pode ser descartada.

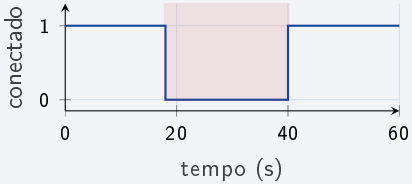
Experimento 1 — indisponibilidade do broker

QUESTÃO

O que acontece com a telemetria quando o broker fica indisponível?

- **Operação normal:** publisher e consumer alinhados na mesma taxa.
- `broker-down` desconecta tudo; `broker-up` restaura e o cliente reconecta sozinho.

```
make broker-down  ... make broker-up
```



exemplo ilustrativo, não medições reais

Reconectar não é o mesmo que recuperar as mensagens perdidas na queda.

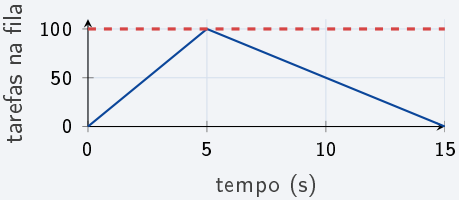
Experimento 2 — sobrecarga do consumidor

QUESTÃO

O que acontece se a entrada supera a capacidade de processamento?

- Worker fixo em ∼10 msg/s; fila interna com 100 posições.
- Rajada eleva a publicação para ∼30 msg/s: a fila cresce e, no limite, descarta (*load shedding*).

```
make burst
```



modelo idealizado, não medições reais

Enquanto $\lambda > X$, a fila cresce até o limite — depois, descartar é *load shedding*, não *backpressure*.

Quadro de síntese

Aula	Pergunta respondida
Segurança em MQTT	Podemos confiar na origem e na integridade das mensagens?
Filas e escalabilidade	Conseguimos processar a carga de maneira eficiente?
Observabilidade e resiliência	Conseguimos detectar falhas e manter ou recuperar o serviço?

Um sistema pronto para produção não é aquele que nunca falha. É aquele cujas falhas podem ser **detectadas**, **compreendidas** e **tratadas**, com garantias de recuperação compatíveis com os requisitos da aplicação.

Observabilidade fornece evidências; mecanismos de resiliência permitem reagir.

Referências

- A. Avizienis, J.-C. Laprie, B. Randell, C. Landwehr. *Basic Concepts and Taxonomy of Dependable and Secure Computing*. IEEE Transactions on Dependable and Secure Computing, 1(1), 2004.
- OpenTelemetry. *Observability Primer*: logs, métricas e traces. https://opentelemetry.io/docs/concepts/observability-primer/
- Prometheus. *Metric Types*: counter, gauge, histogram. https://prometheus.io/docs/concepts/metric_types/
- J. D. C. Little. *A Proof for the Queuing Formula: $L = \lambda W$*. Operations Research, 9(3), 1961.
- OASIS. *MQTT Version 3.1.1 / 5.0 Specification* — níveis de QoS. https://docs.oasis-open.org/mqtt/
- Eclipse Foundation. *Eclipse Paho MQTT Python Client* — reconexão e *callbacks*. https://eclipse.dev/paho/
- M. T. Nygard. *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2018.



Obrigado!



opaiot@icmc.usp.br

+55 (16) 99962-7637

MONITORIA

OpAIoT
Optimization • AI • IoT